

Bangladesh Digital Assistant — Complete Architecture & Deployment-Ready Implementation Blueprint

Document status: Architectural recommendation (not deployed, not implemented)

Version: 1.0

Date: 2026-08-23

Constraint: Zero paid AI/API recurring cost; hosting + domain only

Table of Contents

1. [Executive Summary](#)
2. [Product Goals](#)
3. [Non-Goals](#)
4. [Assumptions and External Dependencies](#)
5. [Decision-Critique Summary](#)
6. [Core Design Principles](#)
7. [System Architecture](#)
8. [Component Architecture](#)
9. [AI Architecture](#)
10. [Agent Architecture Decision](#)
11. [Local AI Stack](#)
12. [RAG Architecture](#)
13. [Knowledge Architecture](#)
14. [Data Ingestion Architecture](#)
15. [Web Crawling Architecture](#)
16. [Bengali / English / Banglish Architecture](#)
17. [Entity Recognition](#)
18. [Document Intelligence](#)
19. [Procedure Engine](#)
20. [Checklist Engine](#)
21. [Source Authority System](#)
22. [Freshness System](#)
23. [Citation and Provenance](#)
24. [Search Architecture](#)
25. [Database Architecture](#)
26. [Admin Architecture](#)
27. [Security Architecture](#)
28. [Privacy Architecture](#)
29. [API Architecture](#)
30. [Frontend and UX Architecture](#)
31. [Geospatial and Local Search](#)
32. [Real-Time Information Policy](#)
33. [Financial Information Policy](#)

- 34. Infrastructure Architecture
- 35. Cost and Resource Architecture
- 36. Testing and Evaluation
- 37. Observability
- 38. Deployment-Readiness Specification
- 39. MVP Scope
- 40. Future Roadmap
- 41. Repository Structure
- 42. Documentation Requirements
- 43. Architecture Diagrams
- 44. Major Risks
- 45. Technical Trade-offs
- 46. Final Recommended Stack
- 47. Final Architecture Decision
- 48. Dependency-Aware Implementation Order

1. Executive Summary

Bangladesh Digital Assistant (BDA) is a **Bangladesh-focused digital information and action layer** — not a generic chatbot. The verified knowledge layer is the source of truth; a locally hosted LLM acts only as interpreter, classifier, planner, summarizer, and conversational interface.

The system is designed as a **modular monolith**: one deployable backend, one PostgreSQL database, one web frontend with integrated admin, optional Redis for cache/queue, and self-hosted open-source AI components. It answers citizen questions about government services, procedures, documents, fees (when evidenced), offices, and geography — always with citations, confidence levels, and conflict warnings.

MVP focus: Passport, NID, birth registration, BRTA (driving licence), TIN/basic tax, government service discovery, Bangladesh geography, official forms/links, and basic education information.

Critical principle: The AI never invents URLs, fees, documents, or procedures. When evidence is insufficient, the system refuses confident answers and falls back to deterministic search/browse behavior.

2. Product Goals

Goal	Description
Bangladesh specificity	Intent, entities, geography, agencies, and sources scoped to Bangladesh
Authoritative retrieval	Prefer Tier 1–2 official sources; never silently merge conflicts
Multilingual UX	Bangla, English, Banglish input and appropriate output
Structured outputs	Checklists, procedure steps, eligibility, verified links
Evidence traceability	Every factual claim linked to source version + verification date
Zero AI/API cost	Self-hosted models; no metered third-party APIs for core functionality

Goal	Description
Deployment-ready repo	Complete config, migrations, tests, runbooks — no unresolved architecture
Modular monolith	Clean boundaries enabling future extraction without premature microservices

3. Non-Goals (MVP)

- Submitting applications on behalf of users
- Accessing government accounts or user credentials
- Executing payments or MFS transactions
- Real-time appointment/slot availability without official free source
- Reliable local business listings without maintained data
- High-risk legal interpretation or guaranteed document authenticity
- Paid LLM/embedding/OCR/translation/search/maps APIs
- Mobile native apps (web-first responsive only)
- WhatsApp/Telegram bots (future scope)

4. Assumptions and External Dependencies

Label	Item
Assumption	Official Bangladesh government websites exist but may be unstable, JS-heavy, or inconsistently structured
Assumption	No free official APIs exist for most services; ingestion is crawl + manual curation
Assumption	Bengali OCR quality on scanned PDFs is imperfect; human review required for high-risk facts
External dependency	Hugging Face / model registries for one-time model download (no recurring API cost)
External dependency	BBS / LGED / official gazette sources for geography seed data (free, manual curation)
External dependency	OpenStreetMap data (ODbL license) for future geospatial features — not MVP-critical
Unsupported capability	Real-time fee/availability from portals without scrapeable public pages
Future integration	Official APIs if/when published by agencies

5. Decision-Critique Summary

All major decisions use the mandated format below. A consolidated final list appears in [Section 47](#).

Decision: Architecture Pattern

Options Considered: Modular monolith, microservices, serverless (Lambda/Cloud Functions), modular monolith + separate worker process

Evaluation Criteria: Operational complexity, zero-cost hosting, data sovereignty, team size fit, failure isolation, future extraction path, Bengali AI pipeline cohesion

Comparison:

- Microservices multiply deployment, networking, and observability cost — unjustified at MVP scale
- Serverless adds cold-start latency for LLM-adjacent workloads and complicates long-running crawl jobs
- Modular monolith keeps one codebase, one DB transaction boundary, shared domain model
- Separate worker process (same repo) handles crawl/OCR without splitting services

Rejected Options:

- **Microservices:** Premature; increases ops burden 3–5× without scale justification
- **Pure serverless:** Poor fit for local LLM sidecar, long crawls, pgvector
- **Single unstructured monolith:** Lacks extraction boundaries

Final Choice: **Modular monolith** with optional **worker process** (same codebase, `worker` entrypoint)

Revisit Conditions: Sustained >500 RPS on chat, independent scaling need for crawl fleet (>50 concurrent domains), or regulatory requirement to isolate document processing

Decision: Backend Framework

Options Considered: FastAPI (Python), Django + DRF, Node.js (NestJS), Go (Fiber/Gin)

Evaluation Criteria: AI/ML ecosystem, async job integration, OpenAPI, Bengali NLP libraries, team velocity, type safety, ORM maturity

Comparison:

- FastAPI: Best Python ML integration (transformers, sentence-transformers, spaCy), async-native, OpenAPI auto-gen
- Django: Heavier; admin built-in but duplicates custom admin needs; slower async story
- Node.js: Weaker local ML inference ecosystem; would require sidecar for all AI
- Go: Excellent performance but poor ML library ecosystem

Rejected Options: Django (admin overlap, heavier), Node (ML friction), Go (ML friction)

Final Choice: **FastAPI 0.115+** with **SQLAlchemy 2.x + Alembic + Pydantic v2**

Revisit Conditions: If team is exclusively TypeScript and accepts AI sidecar-only architecture

Decision: Frontend Framework

Options Considered: Next.js (App Router), React SPA (Vite), Vue 3, SvelteKit, SSR-only (HTMX + Jinja)

Evaluation Criteria: Bangla RTL/bidi support, i18n, mobile performance, admin UI reuse, SEO for public service pages, low-bandwidth, accessibility

Comparison:

- Next.js: SSR/SSG for service discovery pages (SEO), React ecosystem, next-intl for i18n
- Vite SPA: Simpler but weaker SEO for public service catalog
- HTMX: Fast but poor complex admin UX

Rejected Options: Vue/Svelte (smaller Bangla component ecosystem), pure SPA (SEO gap)

Final Choice: Next.js 15 (App Router) + React 19 + TypeScript — monorepo frontend with `/admin` route group (same app, RBAC-gated)

Revisit Conditions: If SEO is deprioritized and team prefers lighter Vite SPA

Decision: Database

Options Considered: PostgreSQL, SQLite (prod), MySQL/MariaDB, MongoDB

Evaluation Criteria: FTS, pgvector, JSONB, transactional integrity, backup tooling, zero cost

Final Choice: PostgreSQL 16+ (production); **SQLite** (local dev/tests only via `DATABASE_URL` switch)

Rejected: MongoDB (weak relational model for procedures/checklists), MySQL (weaker FTS/pgvector story)

Revisit Conditions: >100M chunks with specialized vector DB need — extract retrieval module only

Decision: Search

Options Considered: PostgreSQL FTS + pgvector hybrid, Elasticsearch/OpenSearch, SQLite FTS5, dedicated vector DB (Qdrant, Milvus)

Evaluation Criteria: Zero cost, hybrid keyword+semantic, operational simplicity, Bengali tokenization support

Final Choice: PostgreSQL FTS (simple config + custom Bengali normalization) + pgvector (HNSW) hybrid retrieval with Reciprocal Rank Fusion (RRF)

Rejected: Elasticsearch (extra JVM service, ops cost), dedicated vector DB (unnecessary at MVP scale)

Revisit Conditions: >5M chunks with search latency p95 >500ms after index tuning

Decision: Vector Storage

Options Considered: pgvector in PostgreSQL, separate Qdrant/Weaviate, store embeddings in object storage

Final Choice: pgvector column on `knowledge_chunks.embedding vector(1024)` with HNSW index

Revisit Conditions: Vector index size exceeds 80% RAM on DB host

Decision: Cache and Queue

Options Considered: Redis, PostgreSQL LISTEN/NOTIFY + job table, RabbitMQ, none

Evaluation Criteria: Rate limiting, session cache, job queue, zero cost, simplicity

Final Choice: Redis 7 — cache (LLM response cache keys, rate limits), Celery broker/backend, optional result backend

Rejected: RabbitMQ (extra service without advantage), DB-only queue (insufficient for crawl backpressure)

Revisit Conditions: Redis memory pressure — split cache vs queue instances

Decision: LLM

Options Considered: Qwen2.5-7B-Instruct, Llama 3.1 8B, Gemma 2 9B, Phi-3, Mistral 7B, smaller Qwen2.5-3B/1.5B CPU fallback

Evaluation Criteria: Bengali/Banglish quality, license, CPU Q4 quantization viability, context length, hallucination tendency, self-host

Model	Bengali	CPU Q4	License	Context
Qwen2.5-7B-Instruct	Good	Marginal (16GB+)	Apache 2.0	32K
Llama 3.1 8B	Moderate	Marginal	Llama license	128K
Qwen2.5-3B-Instruct	Acceptable	Yes (8GB)	Apache 2.0	32K

Final Choice:

- **Primary:** Qwen/Qwen2.5-7B-Instruct via llama.cpp server (GGUF Q4_K_M) or Ollama (self-hosted, no API cost)
- **CPU fallback:** Qwen/Qwen2.5-3B-Instruct Q4
- **Inference interface:** OpenAI-compatible local HTTP (`LLM_BASE_URL=http://llm:8080/v1`)

Failure behavior: Deterministic template responses from structured DB; no generative prose

Revisit Conditions: Better Bengali fine-tune available under Apache 2.0 with benchmark improvement >15% on BDA eval set

Decision: Embedding Model

Options Considered: BGE-M3, multilingual-e5-large, paraphrase-multilingual-MiniLM-L12-v2, LaBSE

Final Choice: BAAI/bge-m3 (1024-dim, multilingual including Bengali) via **sentence-transformers**, CPU inference with batching

Fallback: intfloat/multilingual-e5-base (768-dim) if memory constrained — requires re-embedding migration

Revisit Conditions: Embedding memory >4GB RSS on worker — downgrade to MiniLM with acceptance of retrieval quality drop

Decision: Reranker

Options Considered: bge-reranker-v2-m3, cross-encoder ms-marco, deterministic RRF only

Evaluation Criteria: CPU latency, Bengali support, MVP complexity

Final Choice: MVP: Deterministic reranking — authority score × freshness × entity match × RRF fusion. **Optional v1.5:** BAAI/bge-reranker-v2-m3 (CPU, top-20 rerank only)

Rejected for MVP: Cross-encoder reranker (adds 200–800ms CPU latency per query)

Revisit Conditions: nDCG@10 below 0.65 on eval set after tuning weights

Decision: OCR

Options Considered: Tesseract 5 (ben+eng), PaddleOCR, OCRmyPDF, EasyOCR

Final Choice: OCRmyPDF + Tesseract 5 (ben, eng traineddata) for PDFs; **PaddleOCR** optional GPU path disabled by default

CPU behavior: Tesseract acceptable for typed docs; scanned Bengali forms flagged low-confidence

MVP scope: Document upload **disabled by default** (FEATURE_DOCUMENT_UPLOAD=false)

Revisit Conditions: PaddleOCR CPU Bengali CER improves >20% in internal tests AND host has 8GB+ spare RAM

Decision: Translation Strategy

Options Considered: NLLB-200 local, MarianMT, translate everything, native multilingual generation only

Final Choice: Native multilingual generation (Qwen2.5) for responses; **no automatic translation of official names/URLs/fees**; retrieval query expansion uses **Banglish normalization + alias dictionary** only — not full translation

Rejected: Auto-translate all crawled content (corrupts official terminology)

Revisit Conditions: Retrieval recall for Bangla queries <70% — add NLLB-200 for query→English expansion at retrieval time only (not for answers)

Decision: Banglish Normalization

Options Considered: Rule-based transliteration, dictionary lookup, LLM normalization, hybrid

Final Choice: Hybrid pipeline:

1. Script detection (Unicode ranges)
2. Rule-based Banglish→Bangla token map (domain dictionary: passport→পাসপোর্ট, nid→এনআইডি, etc.)
3. `indic-transliteration` + custom fuzzy map (RapidFuzz)
4. Optional LLM rewrite **only for intent classification input** (never for stored facts)

Revisit Conditions: Entity linking F1 <0.75 on Banglish eval set

Decision: Language Detection

Options Considered: fastText lid.176, langdetect, cld3, rule-based script detection

Final Choice: Script detection first (Bangla Unicode block vs Latin) + **fastText lid.176.bin** (local, no API) for mixed text

Revisit Conditions: Misclassification rate >5% on eval set

Decision: Crawler

Options Considered: Scrapy, Requests+BS4, Playwright, Selenium, trafilatura

Final Choice: Scrapy 2.11 (scheduled crawls, politeness, pipelines) + **trafilatura** (main content extraction) + **Playwright** (optional, feature-flagged for JS-rendered pages only)

Rejected: Selenium (heavier), pure Requests (no crawl management)

Revisit Conditions: >30% of Tier-1 sources require JS rendering — enable Playwright worker pool

Decision: PDF Extraction

Options Considered: PyMuPDF, pdfplumber, Apache Tika, pdftotext

Final Choice: PyMuPDF (fitz) for text PDFs; fallback to **OCRmyPDF** for scanned

Revisit Conditions: Table extraction quality insufficient — add pdfplumber for table-specific paths

Decision: Browser Automation

Options Considered: Playwright, Selenium, none

Final Choice: Playwright — disabled by default; enabled per-source in source registry (`requires_js: true`)

Rejected for MVP default: Always-on browser automation (resource heavy, crawl complexity)

Decision: Authentication

Options Considered: Self-hosted JWT + refresh, session cookies, Keycloak, Auth0 (paid)

Final Choice: Self-hosted JWT access tokens (15 min) + HTTP-only refresh cookies (7 days) for registered users; **anonymous session UUID** for public chat (rate-limited)

Admin: Separate `admin_users` table, mandatory MFA (TOTP) for `super_admin` and `knowledge_editor`

Rejected: Auth0/Clerk (paid API cost violates constraint)

Revisit Conditions: Enterprise SSO requirement — self-hosted Keycloak as optional module

Decision: Admin Architecture

Options Considered: Separate admin app, Django admin, integrated Next.js `/admin`

Final Choice: Integrated Next.js `/admin` route group + FastAPI admin API namespace `/api/v1/admin/*` with RBAC

Rejected: Django admin (framework lock-in), separate repo (deployment friction)

Decision: Hosting Model

Options Considered: Single VPS, Docker Compose on VPS, Kubernetes, PaaS

Evaluation Criteria: Zero AI API cost, Bangladesh data sovereignty preference, cost, ops complexity

Final Choice: Docker Compose on single VPS (MVP/small) → Compose multi-node (medium: app + DB split) → K3s (large, future)

Recommended MVP host profile: 8 vCPU, 32 GB RAM, 200 GB SSD (runs Qwen2.5-7B Q4 + PostgreSQL + Redis)

Rejected: Managed Kubernetes for MVP (ops overhead)

Revisit Conditions: >1000 daily active users with p95 chat latency SLA breach

Decision: Monitoring

Options Considered: Datadog (paid), Grafana Cloud (paid tier), self-hosted Prometheus+Grafana+Loki

Final Choice: Prometheus + Grafana + Loki + Alertmanager (self-hosted, zero API cost)

Rejected: Datadog/New Relic (paid)

Decision: CI/CD

Options Considered: GitHub Actions, GitLab CI, Jenkins self-hosted

Final Choice: GitHub Actions (free tier for public/OSS) or GitLab CI (self-hosted runner on same VPS if private)

Revisit Conditions: Self-hosted runner needed for integration tests requiring LLM artifacts

Decision: Storage

Options Considered: Local disk, MinIO (S3-compatible self-hosted), PostgreSQL bytea

Final Choice: MinIO (self-hosted) for crawled artifacts, PDFs, model cache; **encrypted temp bucket** for optional document uploads with 24h lifecycle deletion

Rejected: Cloud S3 (metered storage cost — violates spirit of constraint unless user already has fixed-cost hosting)

Decision: Geospatial / Local Search

Options Considered: OSM + self-hosted Nominatim, Overpass API (public), static curated office list, Google Maps (paid)

Final Choice (MVP): Static curated `service_offices` table seeded from official published office lists (manual + crawl). **No live maps in MVP.**

Future v2: Self-hosted Nominatim + OSM Bangladesh extract

Rejected: Google Maps/Mapbox (paid), unverified business listings

Decision: Document Processing

Final Choice: Optional module; **disabled by default.** When enabled: ClamAV scan → MinIO temp → OCR → classification (rules + lightweight local classifier) → field extraction (regex + layout heuristics, not LLM for PII fields) → user confirmation → delete within 24h

Decision: Agent Orchestration

Options Considered: Multi-agent, tool-calling agent, LLM-only, deterministic workflow + LLM

Final Choice: Deterministic workflow orchestrator + constrained LLM stages (see Section 9). No autonomous tool-calling agent.

Rejected: Multi-agent (hallucination + ops risk), LLM-only (unacceptable accuracy), free tool-calling (unsafe browsing)

Decision: Testing Strategy

Final Choice: Pytest (backend), **Playwright** (frontend E2E), **Great Expectations** (data quality), custom **RAG eval harness** with golden dataset in repo

6. Core Design Principles

1. **Knowledge layer = source of truth**; LLM = language interface only
 2. **Evidence before eloquence** — no evidence → no confident fact
 3. **Never invent** URLs, fees, documents, eligibility
 4. **Conflicts are visible** — never silently merged
 5. **Crawled content is untrusted data** — not instructions (prompt injection defense)
 6. **Deterministic fallback always available** when LLM down
 7. **Data minimization** — no unnecessary PII retention
 8. **Modular monolith boundaries** enforced by import linter
 9. **CPU-first, GPU-optional**
 10. **Bangladesh-first** — default country context without asking
-

7. System Architecture

Layer Responsibilities

Layer	Responsibility
Client	Next.js web + admin UI
Web/API	FastAPI gateway: auth, rate limit, OpenAPI
Application/Domain	Services, checklist engine, procedure engine, geography
AI Orchestration	Deterministic pipeline + local LLM adapter
Retrieval	Hybrid search, authority/freshness ranking
Knowledge Management	Source authority, versioning, publication
Ingestion/Crawling	Scrapy, document processing
Persistence	PostgreSQL + pgvector, Redis, MinIO
Background Jobs	Celery workers + beat scheduler

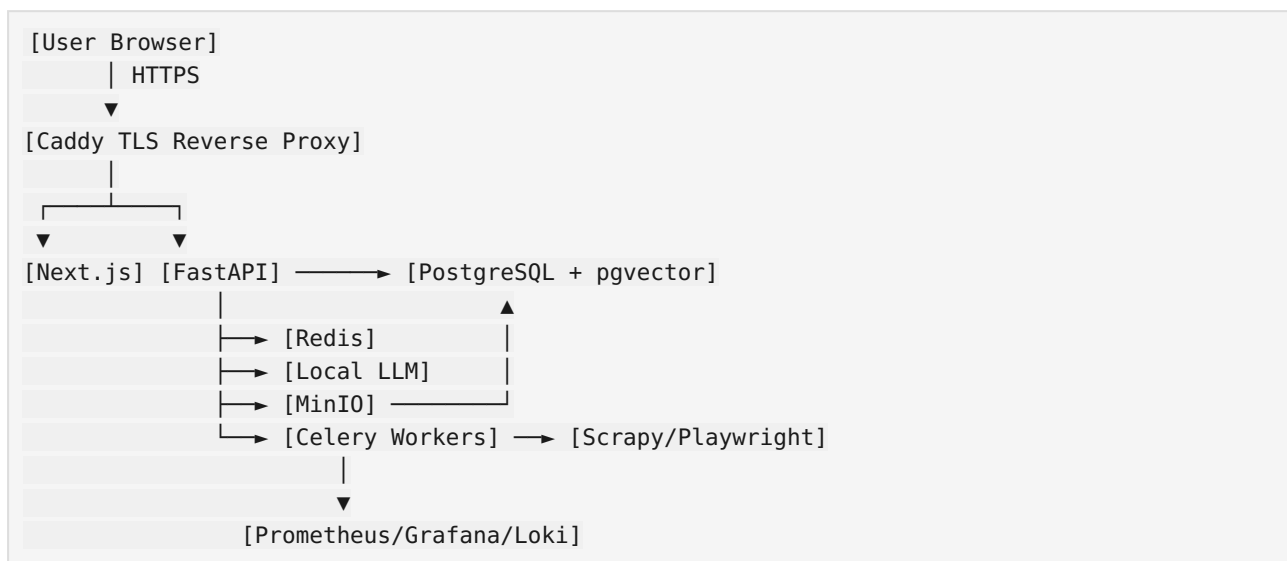
Layer	Responsibility
Administration	Review workflows, RBAC
Observability	Prometheus, Grafana, Loki
Security/Privacy	Auth, audit, PII controls
Infrastructure	Docker Compose, Caddy TLS

Dependency Rules (Allowed → Forbidden)

Module	May Import	Must NOT Import
api	application, ai (interfaces), shared	ingestion crawlers directly
application	domain, persistence interfaces	ai.llm, scrapy
ai	retrieval interfaces, domain types	api, scrapy
retrieval	persistence, knowledge	ai.llm
knowledge	persistence	ai, api
ingestion	knowledge, persistence	api, ai.orchestrator
admin	application, knowledge	—

Enforced via `import-linter` contracts in CI.

High-Level Diagram



8. Component Architecture

Backend Modules (Python packages)

backend/	
├─ app/	
│ └─ main.py	# FastAPI entry
│ └─ api/v1/	# REST routers

```

|   ├── core/                # config, security, logging
|   ├── domain/              # entities, value objects
|   ├── application/         # use cases / services
|   ├── ai/
|   |   ├── orchestrator.py
|   |   ├── pipeline/        # each stage
|   |   └── llm_client.py
|   ├── retrieval/
|   ├── knowledge/
|   ├── ingestion/
|   ├── documents/
|   ├── admin/
|   └── workers/              # Celery tasks

```

Frontend Modules (Next.js)

```

frontend/
├── app/
|   ├── (public)/            # chat, search, services
|   ├── (admin)/admin/       # RBAC admin
|   └── api/                  # BFF optional (minimal)
├── components/
├── lib/i18n/
└── messages/                # bn.json, en.json

```

9. AI Architecture

Controlled Pipeline (Every Chat Request)

Stage	Type	Description
1. Input validation	Deterministic	Length limits, PII pattern warnings, injection heuristics
2. Language detection	Local model + rules	fastText + script detection
3. Banglish normalization	Hybrid	Dictionary + fuzzy + optional LLM for classification path only
4. Intent classification	Local LLM + rules fallback	Taxonomy: procedure_inquiry, document_list, fee_inquiry, office_locator, eligibility, general_info, unsupported
5. Entity extraction	Deterministic NER + gazetteers	Geography, agency, service, form, fee mentions
6. Bangladesh context	Deterministic	Default country=BD; reject non-BD unless explicit
7. Clarification decision	Rules + LLM	Ask only if branch condition unknown (e.g., first vs renewal passport)
8. Query planning	Deterministic	Build retrieval plan: service IDs, filters, required fact types
9. Hybrid retrieval	DB + vector	FTS + pgvector + entity join
	Deterministic	Tier weight × recency × verification

Stage	Type	Description
10. Authority/freshness ranking		
11. Evidence extraction	Deterministic	Select spans from chunks matching fact types
12. Conflict detection	Deterministic	Compare fee/doc/url fields across sources
13. Structured answer planning	Deterministic	Template slots: docs, steps, fees, links, warnings
14. LLM generation	Local LLM	Constrained prompt: only provided evidence JSON; temperature 0.1
15. Citation validation	Deterministic	Every numeric/fee/doc claim must map to evidence ID
16. Confidence calculation	Deterministic	Formula (Section 25)
17. Safety policy	Deterministic	Block payment credentials, impersonation, legal advice
18. Response formatting	Deterministic	Structured JSON → UI components
19. Feedback capture	Async	Store thumbs + optional reason

LLM Constraints (Hard-coded policy)

The LLM **must not**:

- Browse web freely
- Invent URLs/fees/documents
- Override authority ranking
- Treat Tier 5-6 as official
- Execute payments or submissions

Prompt template receives `evidence_bundle` JSON only; system prompt states: "If not in evidence, say you don't know."

Deterministic Fallback (LLM unavailable)

User intent	Fallback behavior
Service lookup	Return <code>services</code> table match + official links
Procedure lookup	Return <code>procedure_steps</code> ordered list
Checklist	Rule engine output from structured fields
Source lookup	Source registry page
Search	Hybrid search results list (no synthesis)
Geography	District/upazila hierarchy browser

10. Agent Architecture Decision

Final Choice: Single deterministic orchestrator + constrained LLM stages (not multi-agent, not tool-calling agent)

Flow: Orchestrator owns state machine; LLM invoked at fixed stages with fixed I/O schemas; retrieval and validation are code, not agent decisions.

Revisit Conditions: Complex multi-step dialogues exceed maintainability of state machine — consider explicit workflow DSL, still not free-form agents

11. Local AI Stack

Component	Model/Tool	License	RAM (CPU Q4)	Bengali	MVP	Fallback
LLM Primary	Qwen2.5-7B-Instruct GGUF Q4	Apache 2.0	~6 GB	Good	Yes	3B or deterministic
LLM CPU-small	Qwen2.5-3B-Instruct Q4	Apache 2.0	~3 GB	Acceptable	Fallback	Templates
Embeddings	bge-m3	MIT	~2 GB	Good	Yes	e5-base
Reranker	RRF + rules	—	—	N/A	Yes	bge-reranker v1.5
Lang detect	fastText lid.176	MIT	~130 MB	OK	Yes	Script rules
OCR	Tesseract ben+eng	Apache 2.0	~500 MB	Moderate	Optional	Disable upload
LLM server	llama.cpp server	MIT	—	—	Yes	Ollama equivalent

Model acquisition: `scripts/models/download_models.sh` — downloads to `models/` volume; checksum verified; air-gap copy supported.

Update process: Register in `model_registry` table → download → benchmark gate → feature flag swap.

12. RAG Architecture

Retrieval Pipeline

```
Query → normalize → expand aliases →  
  ├── FTS branch (PostgreSQL tsvector, weighted)  
  ├── Vector branch (pgvector cosine, top 50)  
  └── Entity branch (service/agency/geo join)  
→ RRF merge (k=60) →  
  authority/freshness/entity boost →  
  top 15 chunks →  
  evidence extraction (fact-type tagged spans)
```

Chunking Strategy

- **Structured facts** (fees, doc lists): stored in relational tables — primary source
- **Narrative content**: chunks of 400–600 tokens, 80-token overlap
- Metadata per chunk: `language`, `agency_id`, `service_id`, `fact_types[]`, `source_tier`, `effective_date`

Evaluation Metrics (MVP targets)

Metric	MVP Target
Recall@10	≥0.80 on golden set
Citation coverage	≥95% for procedural answers
Unsupported-claim rate	<2%
nDCG@10	≥0.70
Banglish intent accuracy	≥0.85

13. Knowledge Architecture

Canonical Entity Model

```
Agency —< Service —< Procedure —< ProcedureStep
      |
      |—< ChecklistItem
      |—< Fee (versioned)
      |—< Form
      |—< ServiceLink (verified URL)
      |—< ServiceOffice >— Geography
```

```
Source —< SourceVersion —< KnowledgeDocument —< KnowledgeChunk — Embedding
```

Domain Coverage

MVP domains: NID, Passport, Birth registration, BRTA/Driving licence, TIN/Tax, Government service discovery, Geography (64 districts), Official forms/links, Basic education (SSC/HSC/university overview)

Post-MVP (schema-ready): Death registration, Citizenship, Police clearance, Vehicle registration, Land, VAT, Trade licence, Immigration, Finance details, Transportation, Commerce

Canonical Service Object

```
service:
  id: uuid
  slug: string          # passport-renewal
  name_bn: string
  name_en: string
  aliases: string[]     # includes banglish
  agency_id: uuid
  category: enum
```

```

status: ACTIVE|OUTDATED|UNDER_REVIEW|CONFLICTED|DISABLED
eligibility: jsonb
required_documents: jsonb
conditional_documents: jsonb
procedures: Procedure[]
fees: Fee[]
forms: Form[]
service_links: ServiceLink[]
offices: ServiceOffice[]
effective_date: date
expiration_date: date | null
last_verified_at: timestampz
confidence: float
review_state: enum
version: int
source_provenance: uuid[]

```

Knowledge Quality Score (KQS)

```

KQS = 0.25×Authority + 0.20×Freshness + 0.15×Completeness
      + 0.15×Verification + 0.10×Consistency + 0.05×RetrievalHitRate
      + 0.05×CitationCoverage + 0.05×(1 - NegativeFeedbackRate)

```

Range 0–100. **Publication rules:**

- KQS ≥70 and no unresolved conflicts → auto-publish narrative chunks
- Fees/docs/eligibility changes → **always manual review** regardless of KQS
- KQS <50 → demote to `UNDER_REVIEW`

14. Data Ingestion Architecture

Pipeline

```

Source Registry → Discovery → Fetch (Scrapy) → Content Extract (trafilatura)
→ PDF/Image branch → Language detect → Normalize text
→ Structure extract (rules + tables) → Metadata → Validate schema
→ Dedupe (simhash) → Chunk → Embed (batch) → Index
→ Change event → Review queue → Publish → Audit

```

Prompt Injection Defense (Crawled Content)

- Strip HTML scripts/forms
- Mark all crawled text `untrusted_content=true`
- Never pass raw HTML to LLM system role
- Sanitize: remove instruction-like patterns ("ignore previous", "system:")

Deduplication

- URL canonicalization + redirect tracking
- Content simhash threshold 0.95 → duplicate
- Keep highest tier source version

15. Web Crawling Architecture

Source Registry Schema

```
source:
  domain: string
  tier: 1-6
  agency_id: uuid | null
  crawl_enabled: bool
  requires_js: bool
  rate_limit_rpm: int          # default 10 for gov sites
  schedule_cron: string
  robots_respected: true
  allow_paths: string[]
  deny_paths: string[]
  parser_profile: string
```

Crawl Schedules

Priority	Tier	Frequency	Examples
P0	1	Daily	Passport, NID, BRTA official
P1	2	Daily	Agency subdomains
P2	3	Weekly	Universities, banks
P3	4-6	Manual/disabled	Media, blogs — ingest only by approval

Legal Compliance

- Respect robots.txt
- Identify crawler: BDABot/1.0 (+https://example.org/bot; contact=...)
- Rate limits enforced per domain
- No authenticated scraping unless explicitly authorized in source registry

16. Bengali / English / Banglish Architecture

Input Processing

```
Input → Script detect → Language detect →
  if banglish: normalize tokens via domain dict →
  if mixed: segment by script →
  expand aliases (BN/EN/Banglish) →
  retrieval query (preserve original for display)
```

Response Policy

User language	Response language
Bangla script	Bangla primary, EN for official form names in parentheses
English	English

User language	Response language
Banglish	Mirror user style; include Bangla for official terms

Never translate: URLs, form IDs, legal citations, fee amounts, agency acronyms (BRTA, NID)

Tokenization

- Bengali: `bnlp` or `indic-nlp-library` for sentence split
- FTS: custom normalization function in PostgreSQL (`unaccent` + bangla digit normalize)

17. Entity Recognition

Gazetteers (Deterministic)

- `geography_gazetteer`: 64 districts, divisions, major upazilas/thanas, city corporations
- `agency_gazetteer`: BRTA, DGFP, DNP&HO, NBR, EC, etc.
- `service_gazetteer`: passport renewal, NID correction, driving licence renewal
- `form_gazetteer`: known form numbers/names

Extraction Pipeline

1. Regex for known patterns (fee amounts ট, dates, form numbers)
2. Dictionary lookup (Aho-Corasick)
3. Fuzzy match (RapidFuzz, threshold 85)
4. Optional LLM NER **for disambiguation only** — outputs must match gazetteer IDs or be rejected

Example: "Mirpur BRTA te driving license renew korte ki lagbe?"

```
{
  "intent": "document_list",
  "service_slug": "driving-licence-renewal",
  "agency_slug": "brta",
  "location": {"type": "area", "name": "Mirpur", "district": "Dhaka", "confidence": 0.82},
  "task": "required_documents",
  "language": "banglish"
}
```

Confirmation threshold: location confidence <0.75 → ask clarifying question

18. Document Intelligence

Default: `FEATURE_DOCUMENT_UPLOAD=false`

When enabled:

```
Upload → size/type check → ClamAV → encrypt → MinIO temp (24h TTL)
→ OCR (Tesseract) → classify (rule-based) → extract fields (regex/templates)
→ show to user for confirmation → optional checklist assist → scheduled delete
```

May do: Extract visible fields, compare to user input, explain labels

Must not do: Authenticate identity, guarantee validity, submit, store >24h, call external APIs

19. Procedure Engine

Workflow Model

```
procedure:
  service_id: uuid
  version: int
  steps:
    - order: 1
      key: eligibility_check
      title_bn / title_en
      description_bn / description_en
      preconditions: jsonb
      required_documents: uuid[]
      fees: uuid[]
      official_url: verified_link_id | null
      responsible_agency_id: uuid
      location_hint: string | null
      estimated_duration: string | null # only if sourced
      dependencies: [step_key]
      conditions: jsonb
      evidence_ids: uuid[]
      last_verified_at: timestampz
      status: active|incomplete|conflicted
```

Branching Example (Passport)

```
Ask: passport_type (e-passport / MRP)
Ask: application_type (renewal / reissue / first)
→ Select procedure branch → generate checklist
```

20. Checklist Engine

Item Types

REQUIRED | OPTIONAL | CONDITIONAL | RECOMMENDED | NOT_APPLICABLE

Engine Logic

1. Load service checklist template from DB
2. Evaluate conditions against user answers (JSON Logic subset)
3. Attach evidence per item
4. Compute item-level confidence = min(evidence confidences)
5. Output structured JSON — **not LLM-generated list**

LLM may **rephrase** checklist for conversation but structured source is DB template.

21. Source Authority System

Tier Definitions

Tier	Description	Default Weight
1	.gov.bd , official gazette	1.0
2	Agency official portals	0.9
3	Universities, banks, official orgs	0.7
4	Recognized institutions	0.5
5	Reliable media	0.3
6	Blogs, forums, UGC	0.1 (never alone for fees/docs)

Conflict Behavior

Fact type	Conflict rule
Fee	Show both with tiers + dates; refuse single value; flag CONFLICTED
Required documents	Union with conflict markers on discrepancies
Application URL	Prefer Tier 1; if tie, prefer newer source_version.fetched_at
Eligibility	Present qualified answer + "verify with office"

Never silently merge contradictory procedural facts.

22. Freshness System

Change Detection Flow

```
Fetch → normalize → sha256(content) → compare to published version
→ if changed: classify impact (LOW/MEDIUM/HIGH)
→ HIGH: create candidate version → validation → mandatory admin approval
→ MEDIUM: auto-approve narrative only; block fee/doc auto-publish
→ LOW: auto-approve with audit log
→ publish → re-index → update last_verified_at
```

High-Risk Fields (Always Manual Review)

Fees, required documents, eligibility, application URLs, office locations, legal requirements, payment instructions

Rollback

source_versions immutable; publish pointer moves; re-index previous version on rollback

23. Citation and Provenance

Provenance Chain

```
AnswerClaim → evidence_span → knowledge_chunk → knowledge_document  
→ source_version → source → crawl_attempt
```

Date Semantics (Never Invent)

Date type	Source
source_published_at	Parsed from page metadata if present
source_updated_at	Parsed if present
system_verified_at	Admin approval or auto-approve timestamp
crawled_at	Fetch timestamp

UI label: "**Last verified by BDA: YYYY-MM-DD**" vs "**Source states updated: unknown**"

Citation Validation

Post-generation validator extracts (fee|document|url|deadline) claims via regex + slot fill comparison to evidence bundle. Mismatch → downgrade confidence to LOW or strip claim.

24. Search Architecture

Final: PostgreSQL FTS + pgvector hybrid

Indexes

```
CREATE INDEX idx_chunks_fts ON knowledge_chunks USING GIN (search_vector);  
CREATE INDEX idx_chunks_embedding ON knowledge_chunks  
  USING hnsw (embedding vector_cosine_ops) WITH (m=16, ef_construction=64);  
CREATE INDEX idx_chunks_service ON knowledge_chunks (service_id, language);
```

25. Database Architecture

Core Tables

Users & Auth: users, user_sessions, refresh_tokens, admin_users, roles, permissions, role_permissions, admin_user_roles

Conversation: conversations, messages, message_citations, message_feedback, clarification_states

Knowledge Core: agencies, services, procedures, procedure_steps, checklist_items, checklist_conditions, fees, forms, service_links, service_offices, sources, source_versions, knowledge_documents, knowledge_chunks

Geography: divisions, districts, upazilas, unions, municipalities, city_corporations, wards, geography_aliases

Operations: crawl_jobs, crawl_attempts, change_events, review_queue_items, audit_logs, feature_flags, model_registry, evaluation_runs

Key Constraints

- `service_links.url` unique per service; must pass domain allowlist trigger
- `fees.amount` requires `evidence_chunk_id` NOT NULL
- Soft delete: `deleted_at` on user-facing entities
- Versioning: `services.version` + published view pointing to active version

Retention Policies

Data	Retention
Anonymous chat	90 days
Registered user chat	Until user deletes + 30 days
Audit logs	2 years
Crawl artifacts	180 days (MinIO lifecycle)
Temp documents	24 hours max

26. Admin Architecture

Roles

Role	Permissions
super_admin	All + user/role management
knowledge_editor	CRUD services/procedures/checklists/sources
reviewer	Approve/reject change events, publish
ops_admin	Crawls, reindex, health, feature flags
auditor	Read-only all + audit logs

Admin Workflows

1. Change review: diff viewer → approve/reject → publish
2. Broken link review: scheduled HEAD checks → queue
3. AI answer review: sample flagged low-confidence chats
4. Feedback triage: incorrect answer reports → link to service version fix
5. Unanswered queries: cluster weekly → content gap report

27. Security Architecture

Threat	Control
Prompt injection	Crawled content sandbox; evidence-only LLM prompts
SSRF (crawler)	Egress allowlist; block RFC1918; no file://
XSS	CSP, React escape, sanitize markdown
SQLi	SQLAlchemy parameterized; no raw SQL in app layer
CSRF	SameSite cookies; CSRF token on admin mutations
Rate limiting	Redis sliding window: anonymous 20/min, auth 60/min
Secrets	Docker secrets / .env not in git; rotate JWT keys
Dependencies	pip-audit, npm audit, Dependabot
Containers	Non-root, read-only rootfs, distroless where possible
PII	Redact NID/passport patterns in logs

Trust Boundaries

```
Internet → Caddy (TLS) → FastAPI → PostgreSQL (private network)
           ↘ MinIO (private)
Crawler workers → egress allowlist only
LLM server → no network egress (inference only)
```

28. Privacy Architecture

- **Data minimization:** Anonymous chat allowed without account
- **Encryption:** TLS 1.3 in transit; PostgreSQL volume encryption at rest
- **Document uploads:** Opt-in consent checkbox; 24h deletion job
- **User deletion:** `DELETE /api/v1/users/me` cascades conversations
- **Logging:** PII scrubber middleware
- **Admin audit:** All publish/approve actions logged immutably

29. API Architecture

Versioning

`/api/v1/*` — breaking changes increment version; maintain v1 for 6 months

Error Format

```
{
  "error": {
    "code": "INSUFFICIENT_EVIDENCE",
    "message": "Human-readable message",
    "correlation_id": "uuid",
    "details": {}
  }
}
```

```
}
}
```

Key Endpoints

Endpoint	Auth	Purpose
POST /api/v1/chat	Optional	Conversational query
GET /api/v1/services	Public	Service catalog
GET /api/v1/services/{slug}	Public	Structured service detail
GET /api/v1/agencies	Public	Agency list
GET /api/v1/districts	Public	Geography
GET /api/v1/search	Public	Hybrid search
GET /api/v1/sources/{id}	Public	Source transparency
POST /api/v1/documents/analyze	Auth + flag	Optional document analysis
DELETE /api/v1/documents/{id}	Auth	Delete temp document
POST /api/v1/feedback	Optional	User feedback
GET /api/v1/health	Public	Liveness
GET /api/v1/readiness	Public	Dependency checks
/api/v1/admin/*	Admin RBAC	Admin operations

POST /api/v1/chat — Request

```
{
  "message": "passport renew korte ki ki lagbe?",
  "conversation_id": "uuid | null",
  "language_preference": "auto | bn | en",
  "clarifications": {"passport_type": "e-passport", "application_type": "renewal"}
}
```

POST /api/v1/chat — Response

```
{
  "conversation_id": "uuid",
  "message_id": "uuid",
  "language": "banglish",
  "confidence": "medium",
  "answer": {
    "summary": "...",
    "checklist": [{"item": "...", "type": "REQUIRED", "evidence_id": "uuid"}],
    "steps": [{"order": 1, "title": "...", "official_url": "https://..."}],
    "fees": [{"amount": "7000", "currency": "BDT", "evidence_id": "uuid"}],
    "warnings": ["Fee information conflicts between sources."],
    "clarifications_needed": []
  },
  "citations": [{
    "evidence_id": "uuid",
    "source_title": "...",
    "source_url": "https://...",

```

```

    "tier": 1,
    "last_verified_at": "2026-03-15T00:00:00Z",
    "excerpt": "...",
  },
  "metadata": {
    "intent": "document_list",
    "service_slug": "passport-renewal",
    "processing_ms": 2340,
    "llm_used": true,
    "fallback_mode": false
  }
}

```

30. Frontend and UX Architecture

Routes

Route	Purpose
/	Landing + chat
/chat/[id]	Conversation
/services	Browse catalog
/services/[slug]	Service detail (SEO)
/districts, /districts/[slug]	Geography browser
/sources/[id]	Source transparency page
/about, /privacy	Static
/admin/*	RBAC admin

Key Components

ChatThread, CitationCard, ConfidenceBadge, ChecklistView, ProcedureSteps, ConflictWarning, ClarificationPrompt, LanguageToggle, ServiceCatalog, AdminDiffViewer, ReviewQueue

State Management

- **Server state:** TanStack Query
- **i18n:** next-intl with `bn`, `en` message files

Degraded States

Condition	UX
LLM down	Banner: "Limited mode — showing verified records only"
Low confidence	Yellow badge + verify warning
Conflict	Red warning + show both sources
No evidence	Transparent unavailable message + search links

31. Geospatial and Local Search

MVP: Curated `service_offices` with address text, `district_id`, optional lat/lon (manually verified)

Query: "Mirpur passport office kothay?" → entity link Mirpur + service → return office records
— **no map embed MVP**

Future v2: Self-hosted Nominatim + OSM tiles

Rejected MVP: Google Maps, Facebook business listings

32. Real-Time Information Policy

Class	Example	MVP Behavior
Static	Required documents	Serve from DB
Periodic	Fees	Crawl schedule + verified date
Near-real-time	Office holiday announcements	Only if crawlable; else unavailable
Live-only	Appointment slot count	Respond: not available

Never fabricate live data.

33. Financial Information Policy

- Show fees **only** with `evidence_chunk_id` and Tier ≤ 2 (Tier 3 for bank fees if official bank site)
 - Payment links from `service_links` table only (domain-verified)
 - Disclaimer: "Confirm fee at official office/portal before payment"
 - **Never** request bKash PIN, bank password, OTP
-

34. Infrastructure Architecture

Docker Compose Services (Production)

```
services:
  caddy:          # TLS reverse proxy
  frontend:       # Next.js
  api:            # FastAPI (uvicorn)
  worker:         # Celery
  scheduler:      # Celery beat
  llm:            # llama.cpp server
  postgres:       # PostgreSQL 16 + pgvector
  redis:
  minio:
  clamav:         # optional, if doc upload enabled
  prometheus:
  grafana:
  loki:
```

Environment Profiles

- `docker-compose.dev.yml` — SQLite or local PG, small models, hot reload
- `docker-compose.staging.yml` — full stack, reduced resources
- `docker-compose.prod.yml` — resource limits, backups, no debug

Reverse Proxy (Caddy)

- Auto TLS (Let's Encrypt)
- Rate limit at edge
- Security headers (HSTS, CSP)

35. Cost and Resource Architecture

Profile	vCPU	RAM	Disk	Notes
Dev laptop	4	16 GB	50 GB	3B model or external dev LLM
Small prod	8	32 GB	200 GB	7B Q4 + full stack
Medium	8+16	32+16	500 GB	App and DB split
Large (future)	scale out	—	—	Read replicas, worker scale

Cost optimization: LLM response cache, embedding batch jobs off-peak, deterministic paths skip LLM

36. Testing and Evaluation

Test Pyramid

Layer	Tool	Coverage target
Unit	pytest	Domain, checklist logic, conflict detection
Integration	pytest + testcontainers	API + DB
Contract	schemathesis (OpenAPI)	All public endpoints
RAG eval	custom harness	Golden dataset 200+ queries
E2E	Playwright	Chat flow, admin publish
Security	OWASP ZAP baseline	CI weekly
Load	k6	50 concurrent chat
a11y	axe-core	WCAG 2.1 AA critical paths

Golden Dataset Samples

Query	Expected intent	Evidence required	Constraints
NID correction korte ki ki lage?	document_list	NID service docs	No invented forms

Query	Expected intent	Evidence required	Constraints
BRTA license renew korbo kivabe?	procedure_inquiry	BRTA renewal steps	Agency=BRTA
TIN certificate kivabe download korbo?	procedure_inquiry	NBR source if present else low confidence	No fake portal
Mirpur passport office kothay?	office_locator	office records	No map API
Nagad theke bank e taka pathale charge koto?	fee_inquiry	Official MFS fee page or refuse	No invented charge
আমার জন্ম নিবন্ধনে নাম ভুল আছে	procedure_inquiry	Birth reg correction	Bangla response

37. Observability

Metrics (Prometheus)

- `bda_api_request_duration_seconds`
- `bda_llm_inference_duration_seconds`
- `bda_retrieval_duration_seconds`
- `bda_answer_confidence` (histogram)
- `bda_citation_validation_failures_total`
- `bda_crawl_success_total`, `bda_crawl_failure_total`
- `bda_broken_links_gauge`
- `bda_knowledge_stale_services_gauge`
- `bda_llm_unavailable_total`

Alerts

- LLM down >5 min → page ops (system degrades gracefully)
- Crawl failure rate >20% → warn
- Broken Tier-1 links → ticket queue
- Citation validation failure rate >5% → block auto-publish

38. Deployment-Readiness Specification

Repository Must Include

- All Dockerfiles with pinned base image digests
- `docker-compose.*.yaml` for dev/staging/prod
- Alembic migrations (complete schema)
- Seed scripts: geography, agencies (skeleton), roles, feature flags
- `scripts/models/download_models.sh`
- `scripts/ingest/` manual import CLI
- `.env.example` with every variable documented
- CI: lint, test, typecheck, migration check, import-linter
- Runbooks
- Eval dataset + `make eval`

- Backup/restore scripts tested in CI

Environment Variables

```
APP_ENV=development|staging|production
APP_SECRET_KEY=
APP_BASE_URL=
CORS_ORIGINS=
DATABASE_URL=postgresql+asyncpg://...
DATABASE_POOL_SIZE=20
REDIS_URL=redis://...
MINIO_ENDPOINT=
MINIO_ACCESS_KEY=
MINIO_SECRET_KEY=
MINIO_BUCKET_ARTIFACTS=
MINIO_BUCKET_TEMP=
LLM_BASE_URL=http://llm:8080/v1
LLM_MODEL_PRIMARY=qwen2.5-7b-instruct
LLM_MODEL_FALLBACK=qwen2.5-3b-instruct
LLM_TIMEOUT_SECONDS=60
LLM_MAX_TOKENS=1024
EMBEDDING_MODEL=BAAI/bge-m3
EMBEDDING_DEVICE=cpu
FEATURE_DOCUMENT_UPLOAD=false
FEATURE_PLAYWRIGHT_CRAWL=false
FEATURE_LLM_ENABLED=true
JWT_SECRET=
JWT_ACCESS_TTL_MINUTES=15
JWT_REFRESH_TTL_DAYS=7
CRAWLER_USER_AGENT=BDABot/1.0 (+https://...)
CRAWLER_EGRESS_ALLOWLIST=*.gov.bd,...
LOG_LEVEL=INFO
PROMETHEUS_ENABLED=true
RATE_LIMIT_ANONYMOUS_PER_MIN=20
RATE_LIMIT_AUTH_PER_MIN=60
```

39. MVP Scope

MVP v1 Domains

Domain	Justification
Passport	Highest citizen demand; well-defined procedures
NID	Universal ID; correction/duplicate queries common
Birth registration	Foundational document; DGFP sources
BRTA (driving licence)	High volume; clear agency
TIN / basic tax	NBR official sources for registration guidance
Government service discovery	Cross-cutting catalog
Geography (64 districts)	Location disambiguation essential
Official forms & links	Verified link registry

Domain	Justification
Basic education (SSC/HSC/university overview)	High query volume; limit to authoritative overview pages

MVP v1 Features

- Multilingual chat with citations and confidence
- Structured service pages (SEO)
- Checklist + procedure engines
- Hybrid search
- Admin review workflow
- Crawler for Tier 1-2 allowlisted domains
- Self-hosted LLM + embeddings
- Feedback collection
- Deterministic fallback mode

MVP v1 Limitations

- No document upload (default off)
- No business listings
- No maps
- No appointment availability
- No payment execution
- Incomplete upazila coverage initially (district-level minimum)
- Bengali OCR not production-grade
- Some services marked UNDER_REVIEW until curated

Acceptance Criteria

1. 200-query eval set: citation coverage $\geq 95\%$, unsupported-claim $< 2\%$
2. All MVP services have ≥ 1 Tier 1-2 source linked
3. Chat functions with LLM disabled (degraded mode)
4. Admin can approve fee change with audit trail
5. p95 chat latency $< 8s$ on small prod profile (with LLM)
6. Security: OWASP ZAP no critical findings
7. Backup restore tested successfully

v1.5

- Document upload + OCR (opt-in)
- bge-reranker
- Expanded upazila/unions
- SSE streaming chat
- Bank fee info (sourced)

v2

- OSM/Nominatim self-hosted
- Local business curated program (manual verification)
- Mobile PWA enhancements
- NLLB query expansion for Bangla retrieval

v3

- Optional messaging integrations (self-hosted)
- Advanced announcement ingestion
- Read replica scaling

40. Future Roadmap

Phase	Objective	Definition of Done
0 Research	Source inventory, domain allowlist	Spreadsheet + tier assignments
1 Foundation	Repo, CI, Docker, module skeleton	CI green
2 Data model	Migrations, geography seed	64 districts seeded
3 Ingestion	Scrapy + manual import CLI	5 agencies ingested
4 Backend API	All v1 endpoints	Contract tests pass
5 Retrieval	Hybrid search + eval ≥ 0.7 nDCG	Eval harness green
6 Local AI	LLM + embedding integration	Fallback tested
7 Frontend	Chat + service pages	E2E pass
8 Admin	Review workflows	Publish fee change E2E
9 Security	OWASP, audit logs	Security checklist
10 Testing	Golden dataset 200+	Metrics hit targets
11 Ops	Runbooks, backup, monitoring	Restore drill pass
12 Scaling prep	import-linter, docs	ADRs complete

41. Repository Structure

Decision: Monorepo

```
bangladesh-digital-assistant/  
├── README.md  
├── LICENSE  
├── docker-compose.yml  
├── docker-compose.dev.yml  
├── docker-compose.prod.yml  
├── .github/workflows/ci.yml  
├── docs/  
│   ├── architecture/  
│   ├── adr/  
│   ├── runbooks/  
│   ├── api/  
│   └── security/  
├── backend/  
├── frontend/  
├── infra/  
└── models/
```

```

├─ data/
│   └─ seeds/
│       └─ gazetteers/
│           └─ evaluation/
├─ importlinter.ini
└─ Makefile

```

42. Documentation Requirements

Document	Path
README	/README.md
ADRs	/docs/adr/NNN-*.md
Local setup	/docs/local-setup.md
Environment reference	/docs/env-reference.md
Deployment guide	/docs/deployment.md
Operations runbook	/docs/runbooks/operations.md
Backup/restore	/docs/runbooks/backup-restore.md
Security model	/docs/security/model.md
Privacy policy draft	/docs/legal/privacy-policy-draft.md
Data source policy	/docs/policies/data-sources.md
Crawler policy	/docs/policies/crawler.md
Model cards	/docs/models/*.md
Evaluation guide	/docs/evaluation.md
Admin guide	/docs/admin-guide.md
API guide	/docs/api/README.md + OpenAPI
Troubleshooting	/docs/troubleshooting.md
Upgrade guide	/docs/upgrade.md
Disaster recovery	/docs/runbooks/disaster-recovery.md
License inventory	/docs/attribution/licenses.md

43. Architecture Diagrams

User Request → Answer Flow

```

User query → API → Orchestrator → [Retrieve + Rank + Validate]
→ LLM (optional) → Citation check → Confidence → JSON response → UI render

```

Admin Workflow

Crawl/Edit → Change Event → Review Queue → Approve/Reject
→ Publish Version → Reindex → Audit Log

Database ERD (Conceptual)

Division 1-* District 1-* Upazila
Agency 1-* Service 1-* Procedure 1-* ProcedureStep
Service 1-* Fee, Form, ServiceLink, ChecklistItem, ServiceOffice
Source 1-* SourceVersion 1-* KnowledgeDocument 1-* KnowledgeChunk

Backup & DR Flow

Nightly: pg_dump → encrypt → MinIO backup bucket → offsite copy (rsync)
Weekly: restore drill to staging
RPO: 24h | RT0: 4h (small prod)

Security Trust Boundaries

Untrusted: User input, crawled web, uploaded docs
Semi-trusted: LLM output (must pass validators)
Trusted: PostgreSQL published records, admin-approved versions

44. Major Risks

Risk	Impact	Mitigation
Government site structure changes	Broken ingestion	Change detection + admin review + degraded mode
LLM hallucination despite guards	Wrong citizen guidance	Evidence-only prompts + citation validator + confidence gating
Bengali retrieval quality	Missed answers	Alias gazetteers + hybrid search + eval set
Host resource insufficient for 7B	Slow/unusable chat	3B fallback + deterministic mode
Legal concern re crawler	Service shutdown	robots.txt, rate limits, allowlist, contact page
Outdated fees published	Financial harm	High-risk manual review always
Prompt injection via crawled pages	Unsafe answers	Sanitize + never execute crawled instructions

45. Technical Trade-offs

Trade-off	Choice	Cost
Accuracy vs speed	Smaller model fallback	Lower fluency in Banglish

Trade-off	Choice	Cost
Monolith vs microservices	Monolith	Later extraction effort
pgvector vs ES	pgvector	Less specialized text analytics
Auto-publish vs manual	Manual for high-risk	Slower freshness for fees
Feature breadth vs quality	8 MVP domains	Many services unavailable initially
Document intelligence vs privacy	Disabled by default	Less "wow" feature

46. Final Recommended Stack

Layer	Choice	Why	License
Frontend	Next.js 15 + React 19 + TS	SEO, i18n, admin unified	MIT
Backend	FastAPI + SQLAlchemy + Alembic	ML ecosystem, OpenAPI	MIT
Database	PostgreSQL 16 + pgvector	Hybrid search, one DB	PostgreSQL
Search	PG FTS + pgvector + RRF	Zero extra service	—
Vector	pgvector HNSW	Co-located with data	—
Cache/Queue	Redis + Celery	Proven, simple	BSD
LLM	Qwen2.5-7B-Instruct (llama.cpp)	Bengali quality, Apache 2.0	Apache 2.0
Embeddings	bge-m3	Multilingual	MIT
Reranker	Deterministic (v1)	CPU budget	—
OCR	OCRmyPDF + Tesseract	Free, optional	Apache 2.0
Crawler	Scrapy + trafilatura	Scalable crawl	BSD
PDF	PyMuPDF	Fast text extract	AGPL
Auth	Self-hosted JWT + TOTP admin	No paid auth	—
Admin	Next.js /admin + FastAPI	Single deployment	—
Storage	MinIO	S3-compatible self-host	AGPL
Proxy	Caddy	Auto TLS	Apache 2.0
Monitoring	Prometheus + Grafana + Loki	Self-hosted	Apache 2.0
CI/CD	GitHub Actions	Standard	—
Testing	pytest + Playwright + k6	Full pyramid	—

47. Final Architecture Decision

Decision	Final Choice
Architecture pattern	Modular monolith + Celery worker process
Backend	FastAPI (Python 3.12)

Decision	Final Choice
Frontend	Next.js 15 App Router (public + <code>/admin</code>)
Database	PostgreSQL 16 + pgvector
Search	PostgreSQL FTS + pgvector hybrid (RRF)
Vector storage	pgvector on <code>knowledge_chunks</code>
Cache and queue	Redis 7 + Celery
LLM	Qwen2.5-7B-Instruct (llama.cpp GGUF Q4); fallback Qwen2.5-3B
Embedding model	BAAI/bge-m3
Reranker	Deterministic authority/freshness/entity (bge-reranker v1.5 optional)
OCR	OCRmyPDF + Tesseract (feature off by default)
Crawler	Scrapy + trafilatura (+ Playwright flag)
PDF extraction	PyMuPDF
Authentication	Self-hosted JWT + refresh cookies; admin TOTP MFA
Admin architecture	Integrated Next.js admin + <code>/api/v1/admin</code>
Hosting strategy	Docker Compose on VPS (8 vCPU / 32 GB small prod)
Monitoring	Prometheus + Grafana + Loki
CI/CD	GitHub Actions
MVP scope	Passport, NID, birth reg, BRTA, TIN, service discovery, geography, forms/links, basic education
Biggest technical risks	Crawled source instability; Bengali RAG quality; LLM hallucination; host RAM limits
Biggest product risks	Outdated official info causes harm; user expects live transactions; incomplete geographic/office data
Must NOT build initially	Payment execution, application submission, paid APIs, business listings, live appointments, always-on OCR uploads, multi-agent autonomy, microservices

48. Dependency-Aware Implementation Order

1. Repository scaffold + CI + Docker dev compose + import-linter
2. PostgreSQL schema migrations (all tables) + geography seed (64 districts)
3. Domain modules: Agency, Service, Source, Geography (CRUD services)
4. Source registry + allowlist + manual JSON import CLI
5. Scrapy project + ingestion pipelines → `knowledge_documents/chunks`
6. Embedding worker + pgvector index + FTS configuration
7. Hybrid retrieval module + ranking weights + eval harness skeleton
8. Procedure + checklist engines (pure deterministic)
9. Entity gazetteers + Banglish normalization pipeline
10. FastAPI public endpoints: services, search, geography, health
11. AI orchestrator pipeline (without LLM) → deterministic answers
12. Local LLM adapter + constrained generation + citation validator
13. Chat endpoint integration + confidence scoring + conflict detection

14. Next.js public UI: chat, service pages, citations, i18n
15. Admin API + RBAC + review queue + publish workflow
16. Next.js admin UI: diff viewer, approvals, crawl triggers
17. Celery schedules: crawls, link checker, embedding batch, temp doc purge
18. Redis rate limiting + auth (JWT) + audit logging
19. MinIO artifact storage + backup scripts
20. Prometheus/Grafana/Loki dashboards + alerts
21. Golden evaluation dataset (200+ queries) + tune retrieval weights
22. Security hardening: OWASP ZAP, SSRF tests, prompt injection tests
23. Runbooks + deployment compose prod + restore drill
24. MVP content curation: 8 domains Tier-1 sources manually verified
25. Feature flags final review + DOCUMENT_UPLOAD=false default
26. Staging acceptance test against MVP criteria
27. Tag v1.0.0-rc – deployment-ready, not deployed

Hard dependencies: (6) requires (5); (11) requires (7,8,9); (12) requires (11); (13) requires (12,10); (14) requires (13); (15-16) requires (4,5); (21) requires (13,24); (26) requires all above.

Label Summary

- **Architectural recommendation:** Entire document
 - **Assumption:** Government sites lack APIs; Bengali OCR imperfect
 - **Future integration:** OSM, messaging, NLLB retrieval expansion, bge-reranker v1.5
 - **Unsupported capability:** Live appointments, payment execution, verified business listings MVP
 - **Do not invent:** No specific `.gov.bd` URLs, fees, or procedures listed as facts — content comes only through ingestion/curation during implementation
-

This blueprint is complete for a coding agent to implement the full deployment-ready repository without unresolved architectural decisions.